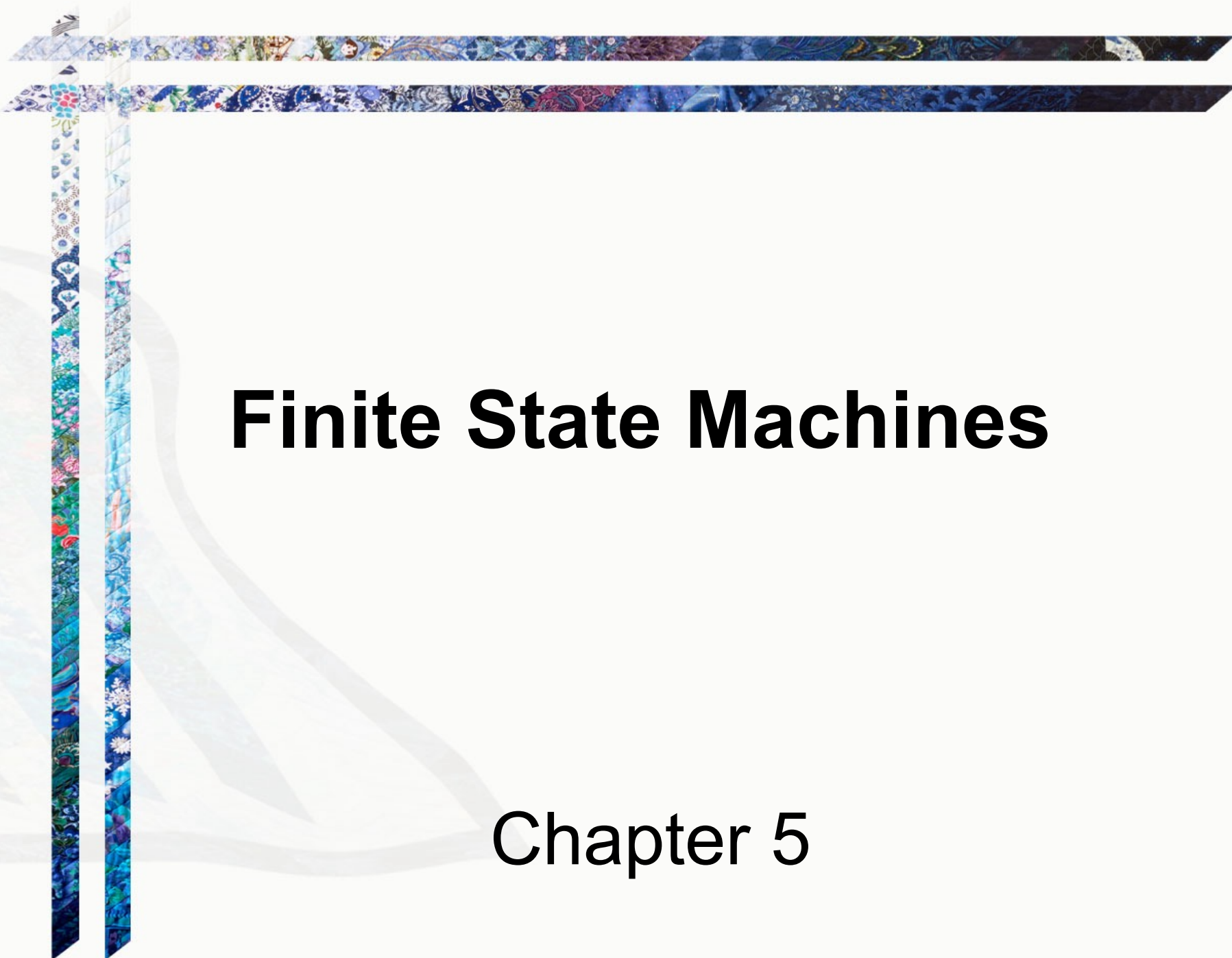
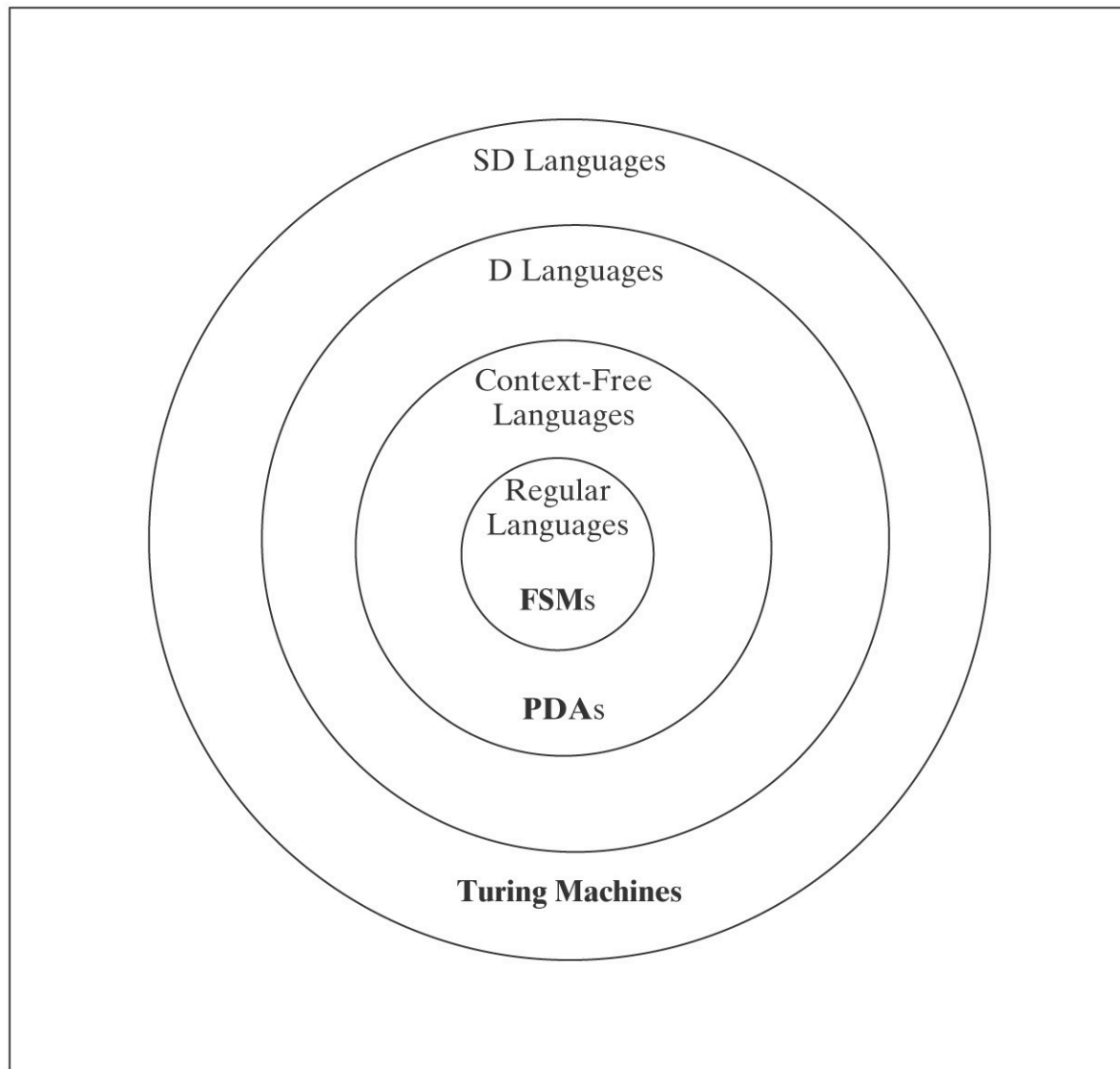




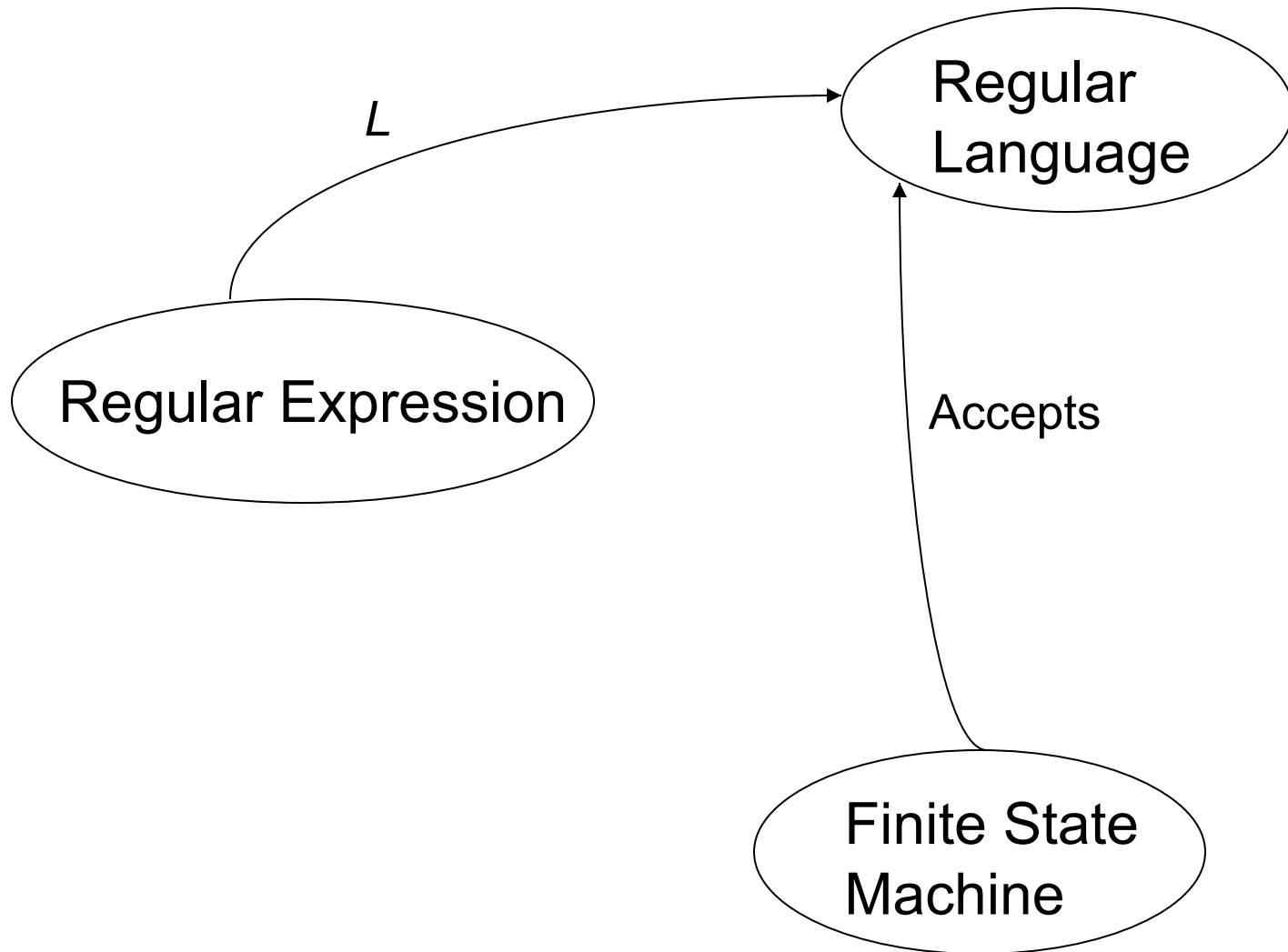
Finite State Machines

Chapter 5

Languages and Machines

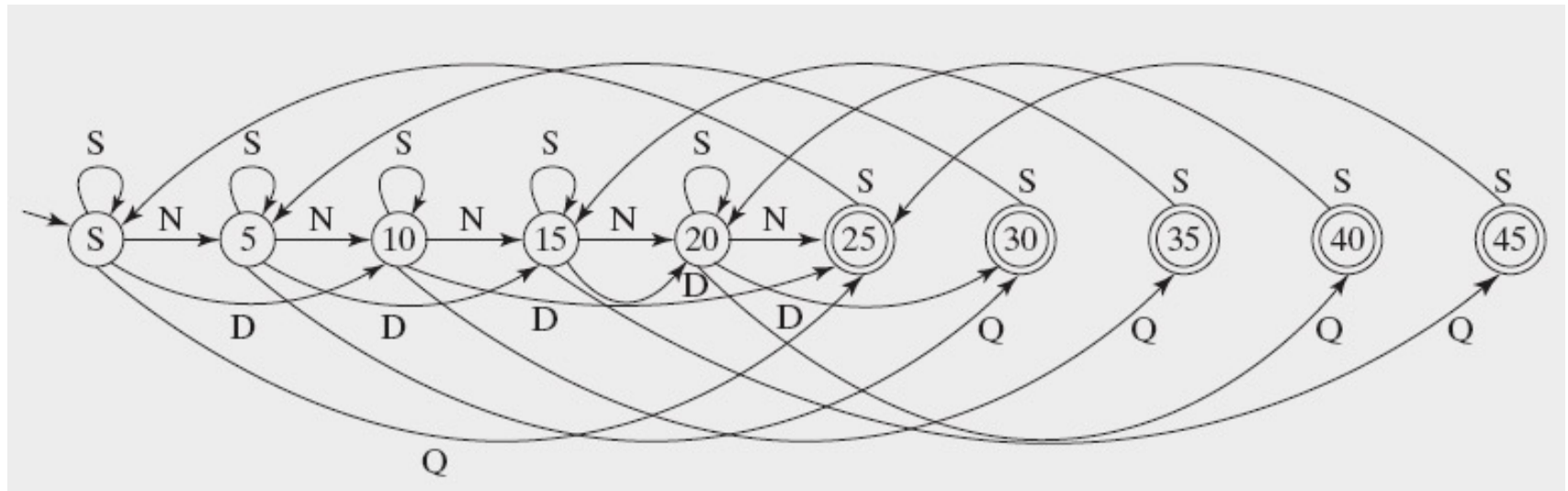


Regular Languages



Finite State Machines

- An FSM for a vending machine:
 - One soda (S) is \$.25
 - No pennies
 - Max credit \$.45





Definition of a DFSM

$M = (K, \Sigma, \delta, s, A)$, where:

K is a finite set of **states**

Σ is an **alphabet**

$s \in K$ is the **initial state**

$A \subseteq K$ is the set of **accepting (final) states**, and

δ is the **transition function** from $(K \times \Sigma)$ to K



Accepting by a DFSA

Informally, M **accepts a string** w iff M winds up in some element of A when it has finished reading w .

The **language** accepted by M , denoted $L(M)$, is the set of all strings accepted by M .



Configurations of DFSMs

A **configuration** of a DFSM M is an element of:

$$K \times \Sigma^*$$

It captures the two things that can make a difference to M 's future behavior:

- its current state
- the input that is still left to read.

The **initial configuration** of a DFSM M , on input w , is:

$$(s, w)$$



The Yields Relations

The *yields-in-one-step* relation $\vdash_M$:

$(q, w) \vdash_M (q', w')$ iff

- $w = a w'$ for some symbol $a \in \Sigma$, and
- $\delta(q, a) = q'$

$\vdash_M^*$ is the reflexive, transitive closure of $\vdash_M$.



Computations Using FSMs

A **computation** by M is a finite sequence of configurations $C_0, C_1, \dots, C_n$ for some $n \geq 0$ such that:

- C_0 is an initial configuration,
- C_n is of the form (q, ε) , for some state $q \in K_M$,
- $C_0 \vdash_M C_1 \vdash_M C_2 \vdash_M \dots \vdash_M C_n$.

Accepting and Rejecting

A DFMSM M **accepts** a string w iff:

$$(s, w) \vdash_M^* (q, \varepsilon), \text{ for some } q \in A.$$

A DFMSM M **rejects** a string w iff:

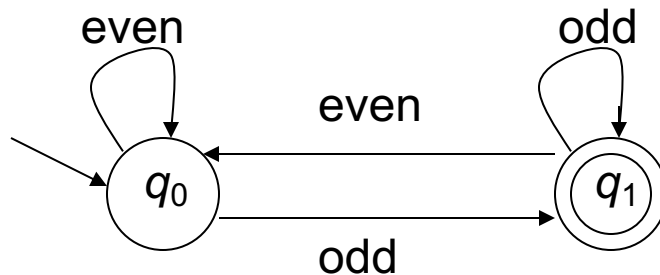
$$(s, w) \vdash_M^* (q, \varepsilon), \text{ for some } q \notin A.$$

The **language accepted by** M , denoted $L(M)$, is the set of all strings accepted by M .

Theorem: Every DFMSM M , on input s , halts in $|s|$ steps.

An Example Computation

An FSM to accept **odd integers**:



On input 235, the configurations are:

$$\begin{array}{lcl} (q_0, 235) & \vdash_M & (q_0, 35) \\ & \vdash_M & \\ & \vdash_M & \end{array}$$

Thus $(q_0, 235) \vdash_M^* (q_1, \varepsilon)$



Regular Languages

A language is *regular* iff it is accepted by some FSM.



A Very Simple Example

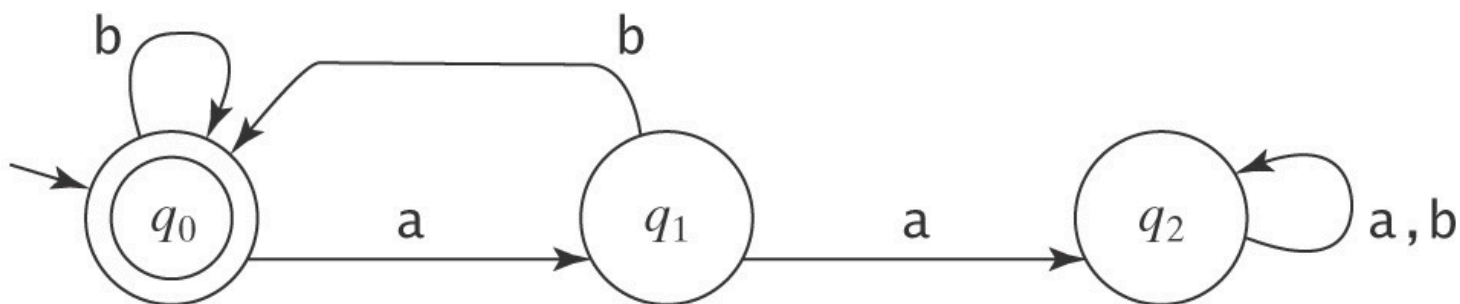
$L = \{w \in \{a, b\}^* :$

every a is immediately followed by a $b\}$.

A Very Simple Example

$L = \{w \in \{a, b\}^* :$

every a is immediately followed by a $b\}$.



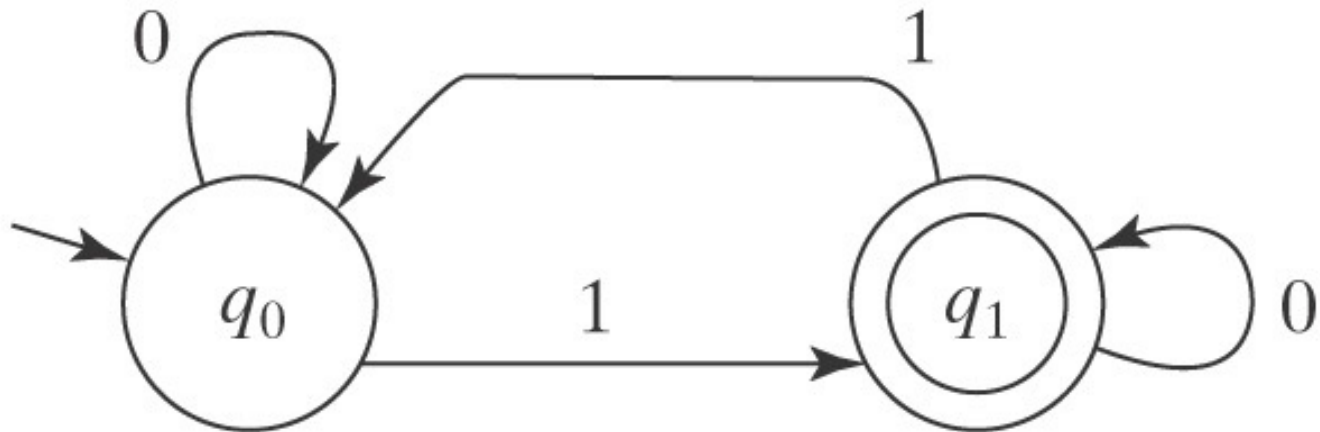


Parity Checking

$L = \{w \in \{0, 1\}^* : w \text{ has odd parity}\}.$

Parity Checking

$L = \{w \in \{0, 1\}^* : w \text{ has odd parity}\}.$

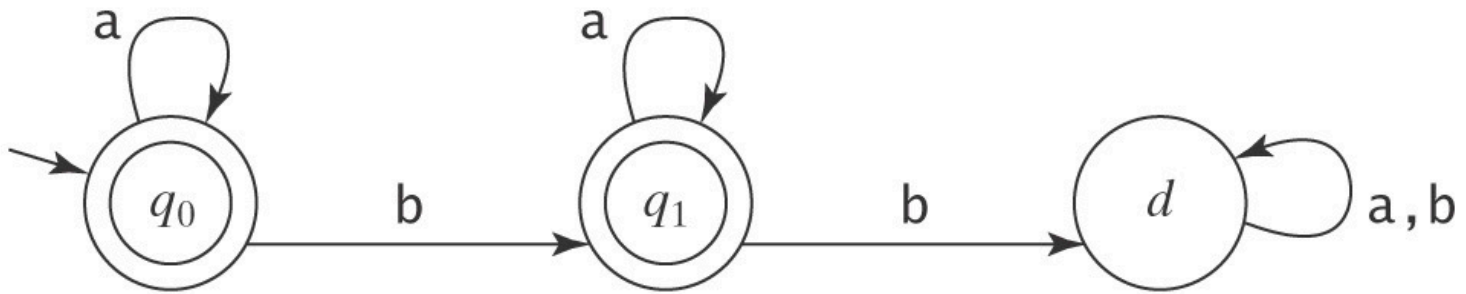


No More Than One b

$L = \{w \in \{a, b\}^* : w \text{ contains at most one } b\}.$

No More Than One b

$L = \{w \in \{a, b\}^* : w \text{ contains at most one } b\}.$





Checking Consecutive Characters

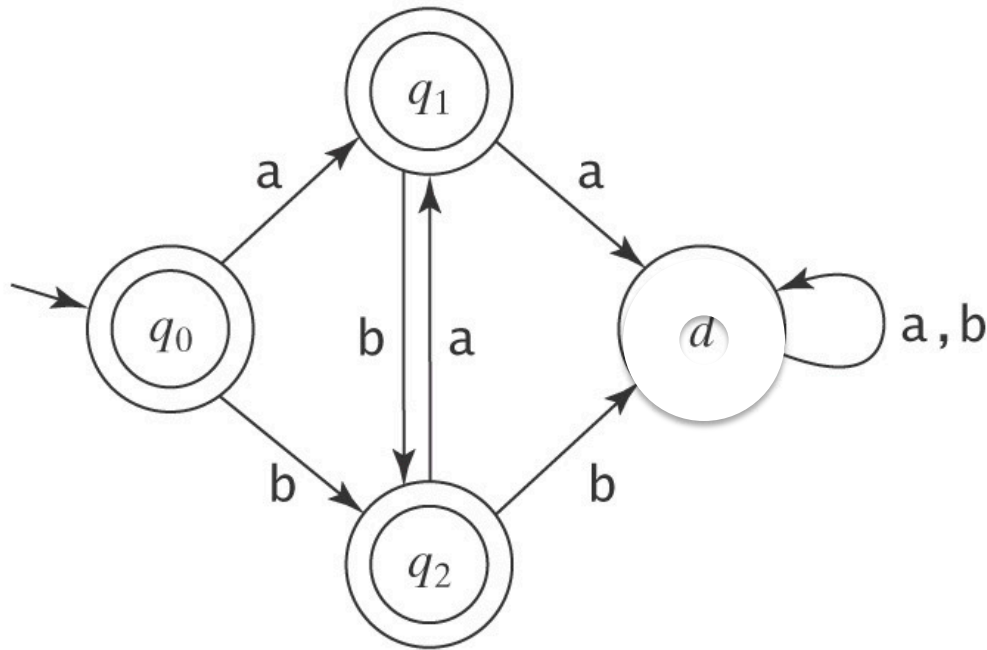
$L = \{w \in \{a, b\}^* :$

no two consecutive characters are the same}.

Checking Consecutive Characters

$L = \{w \in \{a, b\}^* :$

no two consecutive characters are the same}.





Dead States

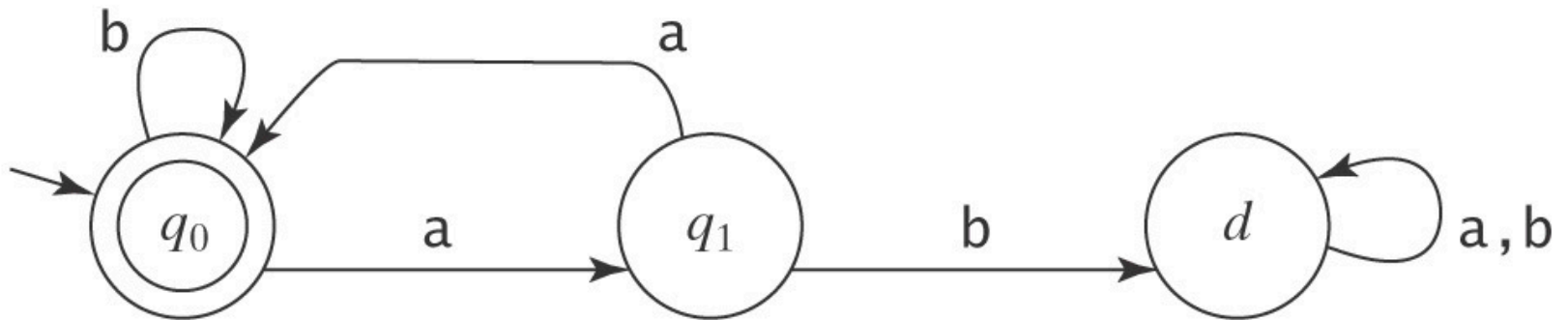
$L =$

$\{w \in \{a, b\}^* : \text{every } a \text{ region in } w \text{ is of even length}\}$

Dead States

$L =$

$\{w \in \{a, b\}^* : \text{every } a \text{ region in } w \text{ is of even length}\}$



Dead States

$L =$

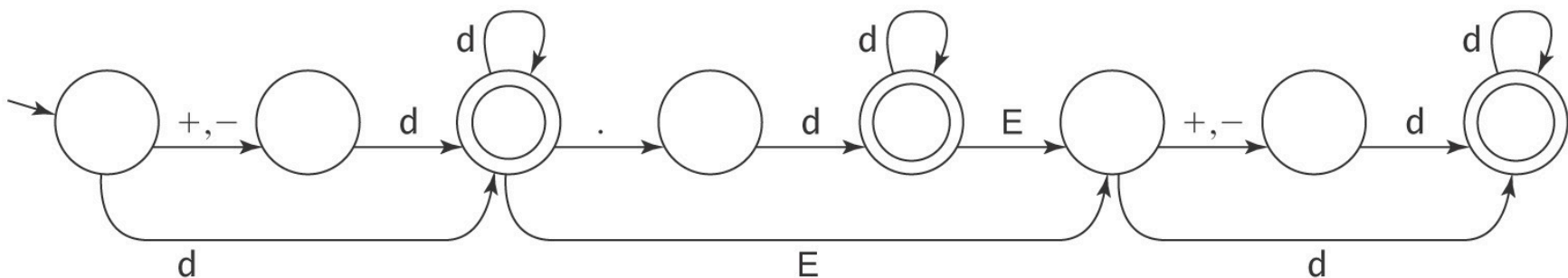
$\{w \in \{a, b\}^* : \text{every } b \text{ in } w \text{ is surrounded by } a's\}$

The Language of Floating Point Numbers is Regular

Example strings:

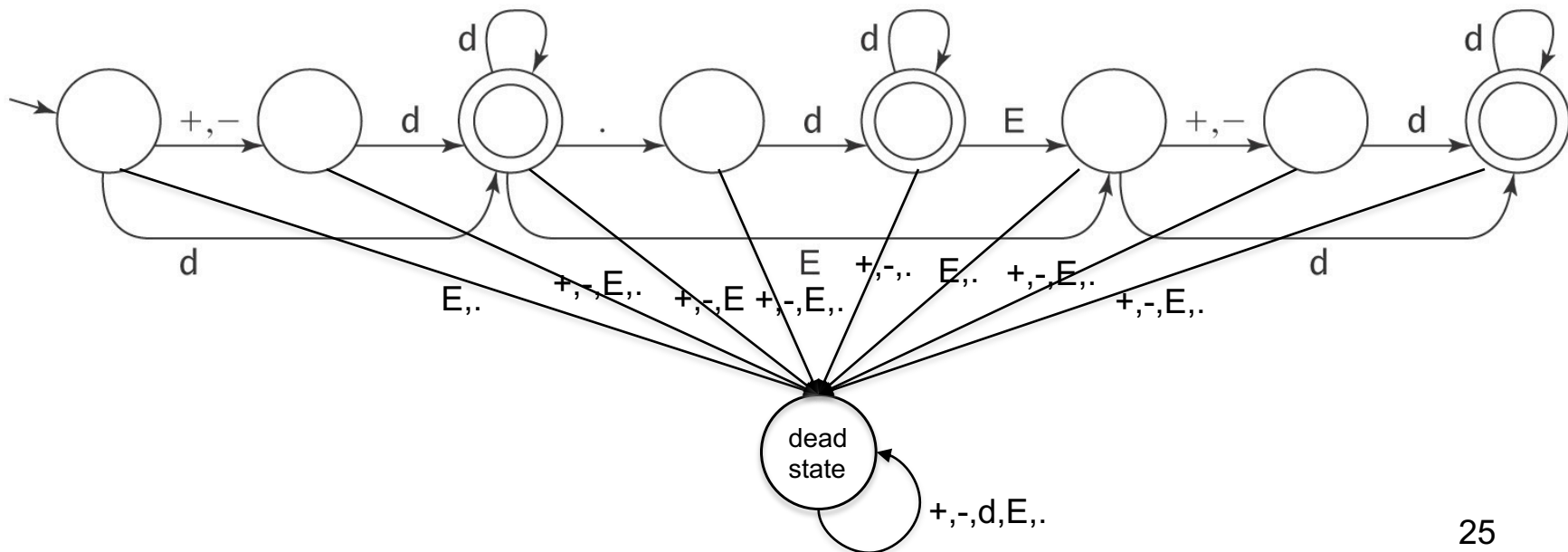
+3.0, 3.0, 0.3E1, 0.3E+1, -0.3E+1, -3E8

The language is accepted by the DFMS:

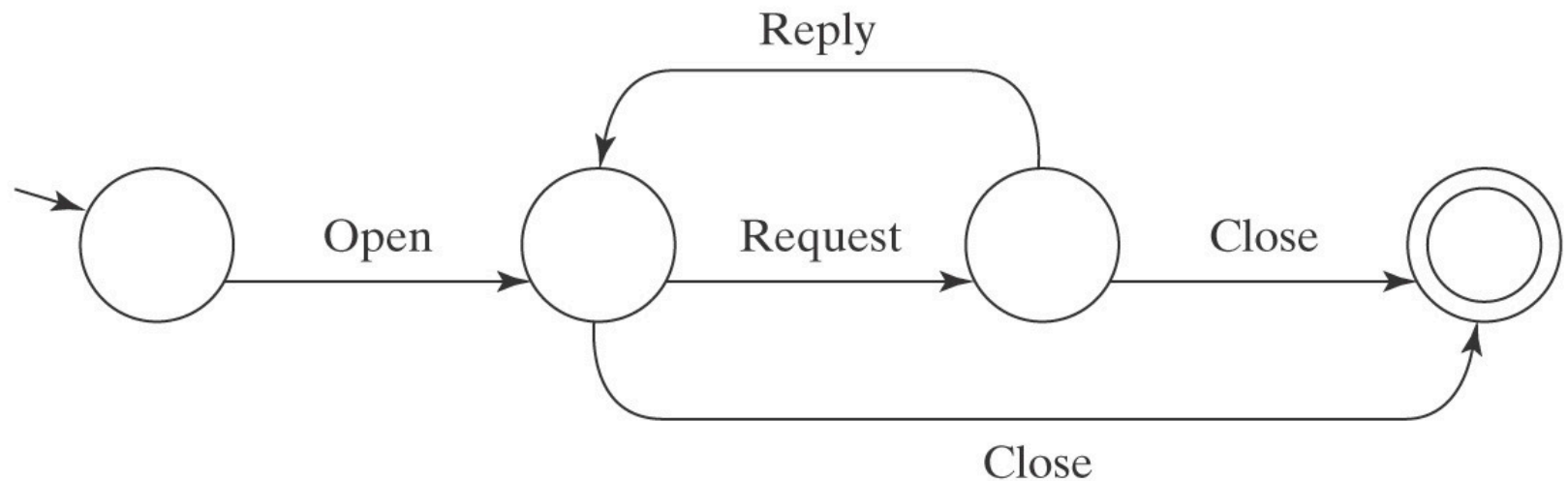


DFSMs are complete

- The transition function is always **complete**
- The missing transitions are leading to a “dead” state
- Often not shown for clarity



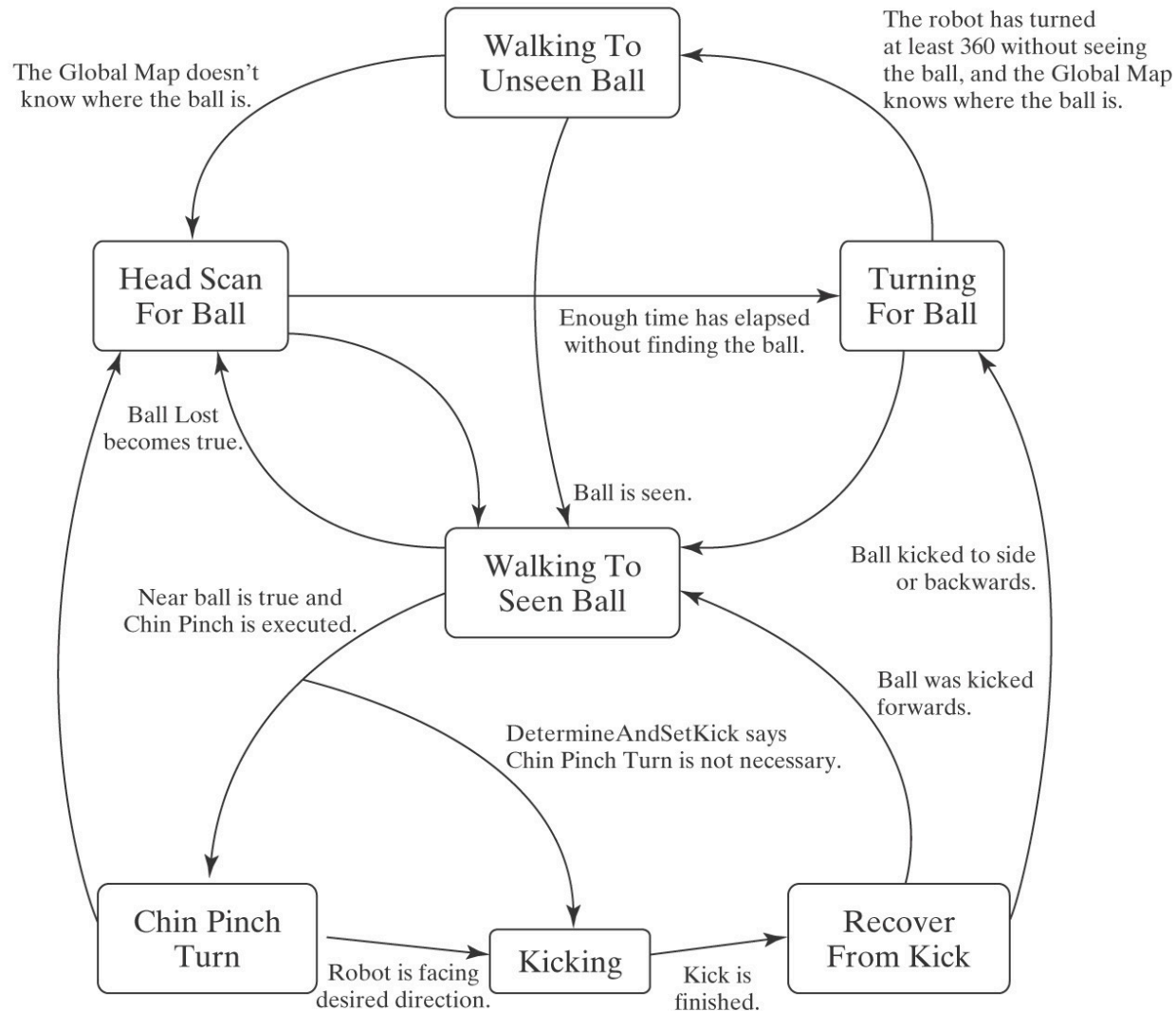
A Simple Communication Protocol



Controlling a Soccer-Playing Robot



A Simple Controller



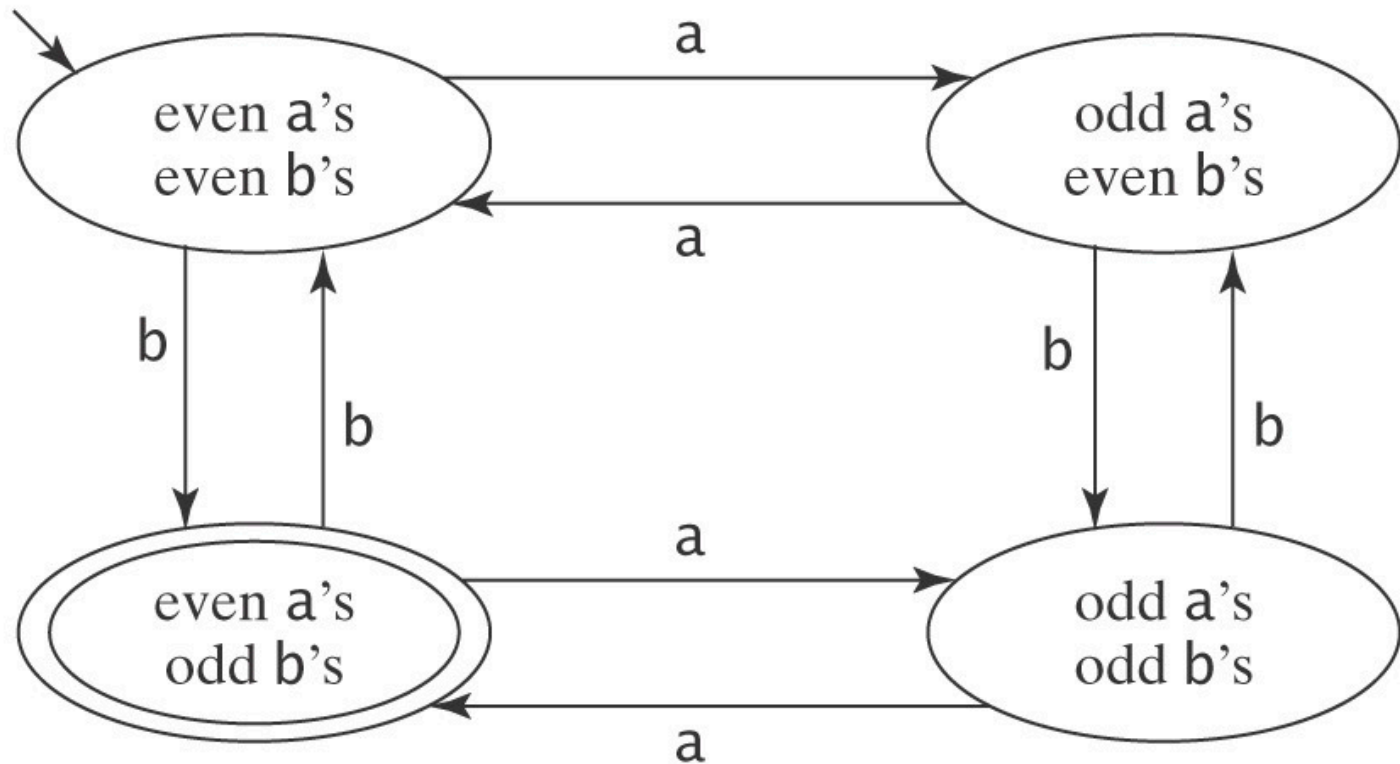


Programming FSMs

Cluster strings that share a “future”.

Let $L = \{w \in \{a, b\}^* : w \text{ contains an even number of } a' \text{ s and an odd number of } b' \text{ s}\}$

Even a's Odd b's



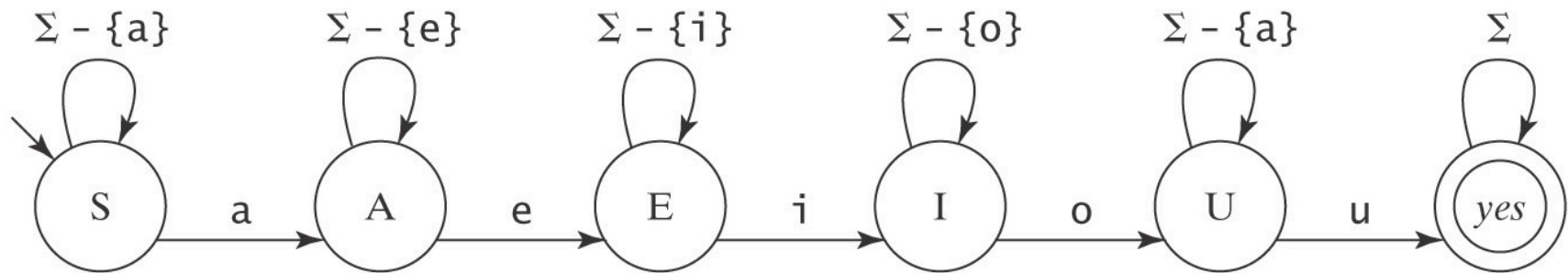


Vowels in Alphabetical Order

$L = \{w \in \{a - z\}^* : \text{all five vowels, } a, e, i, o, \text{ and } u, \\ \text{occur in } w \text{ in alphabetical order}\}.$

Vowels in Alphabetical Order

$L = \{w \in \{a - z\}^* : \text{all five vowels, } a, e, i, o, \text{ and } u, \text{ occur in } w \text{ in alphabetical order}\}.$





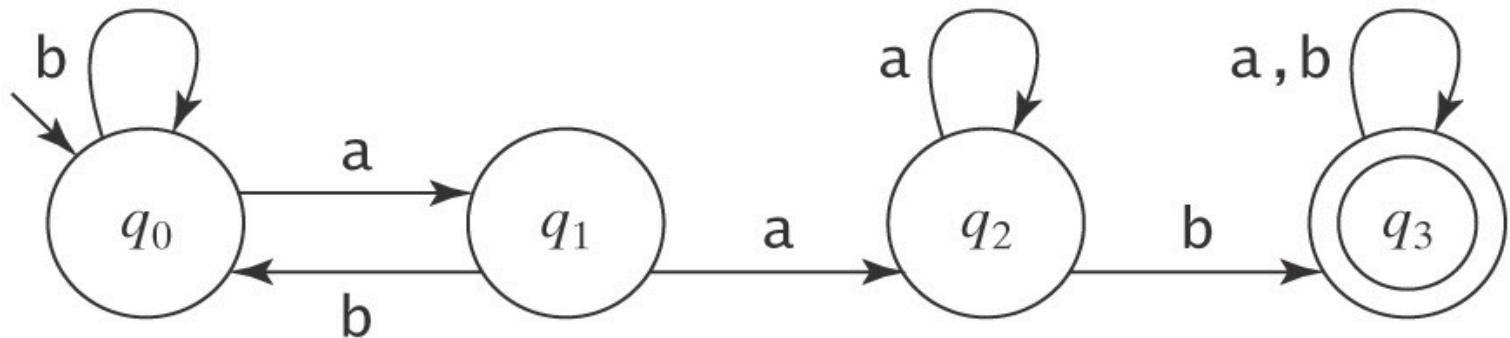
Complementing FSMs

$L = \{w \in \{a, b\}^* : w \text{ does not contain the substring } aab\}.$

Complementing FSMs

$L = \{w \in \{a, b\}^* : w \text{ does not contain the substring } aab\}.$

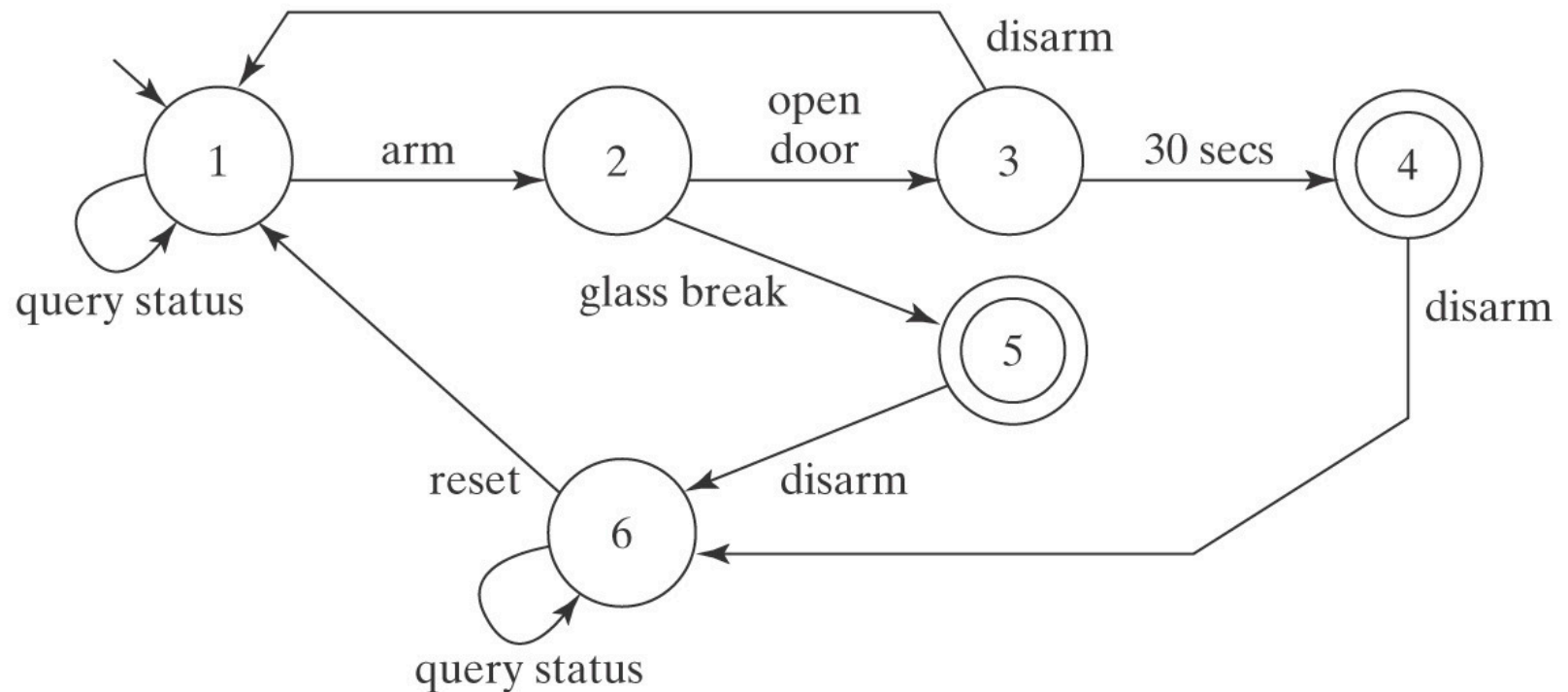
Start with a machine for $\neg L$:



How must it be changed?

A Building Security System

$L = \{\text{event sequences such that the alarm should sound}\}$



FSMs Predate Computers



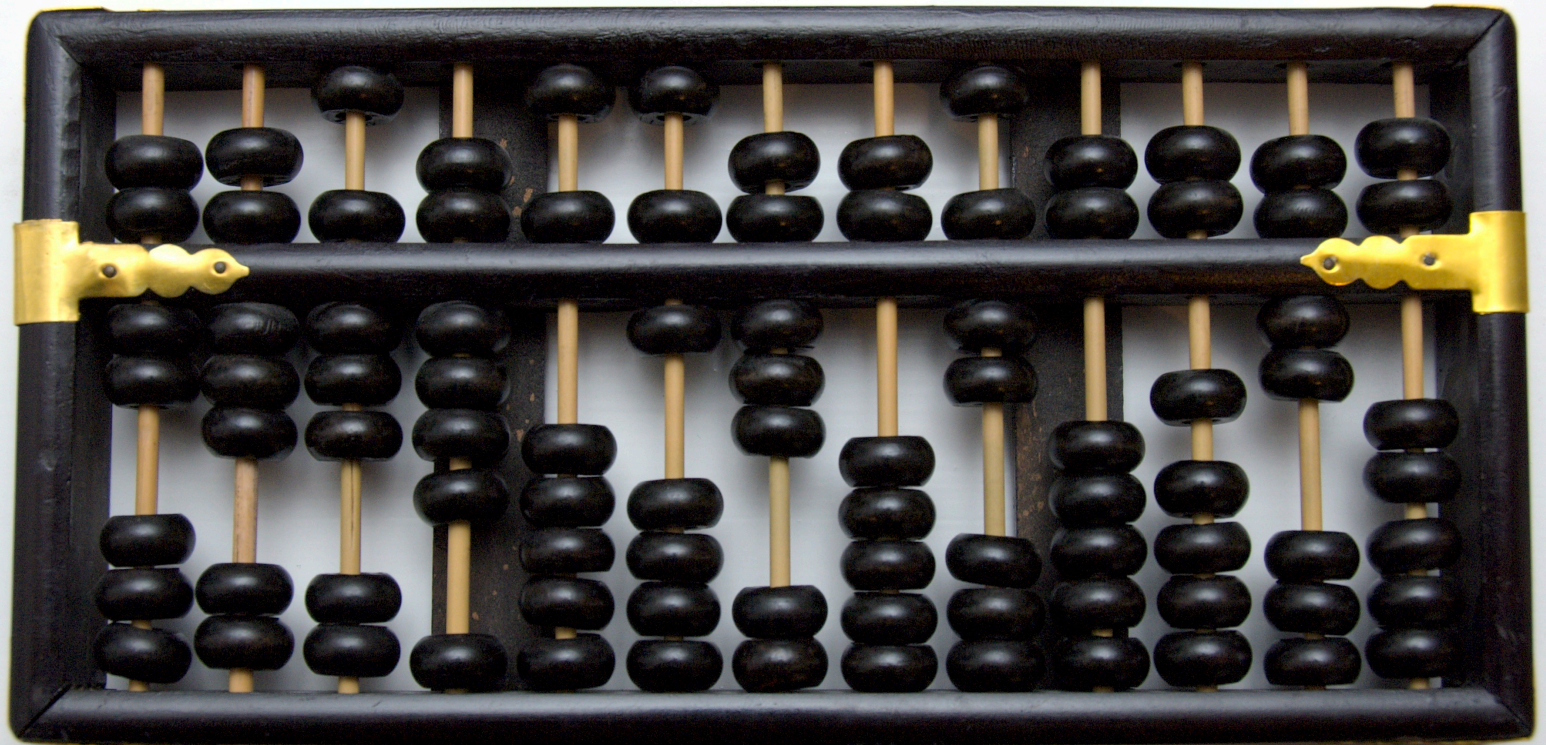
The Antikythera mechanism (Greece, 80 BC)

FSMs Predate Computers



The Prague Orloj, originally built in 1410.

FSMs Predate Computers



The abacus

FSMs Predate Computers



The Jacquard Loom (1801)



The Missing Letter Language

Let $\Sigma = \{a, b, c, d\}$.

Let $L_{Missing} =$
 $\{w : \text{there is a symbol } a_i \in \Sigma \text{ not appearing in } w\}.$

Try to make a DFSA for $L_{Missing}$:

Definition of an NDFSM

A **nondeterministic** FSM (NDFSM) is

$M = (K, \Sigma, \Delta, s, A)$, where:

K is a finite set of **states**

Σ is an **alphabet**

$s \in K$ is the **initial state**

$A \subseteq K$ is the set of **accepting states**, and

Δ is the **transition relation**. It is a finite subset of

$$(K \times (\Sigma \cup \{\varepsilon\})) \times K$$

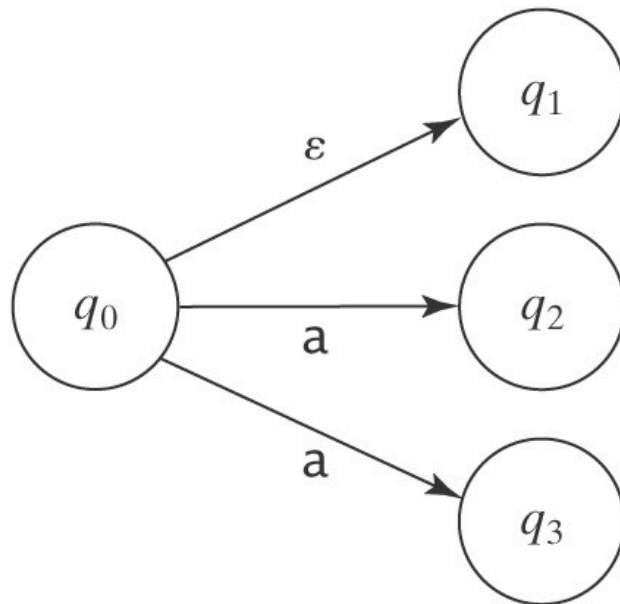


Accepting by an NDFSM

M **accepts** a string w iff there exists *some path* along which w drives M to some element of A .

The **language** accepted by M , denoted $L(M)$, is the set of all strings accepted by M .

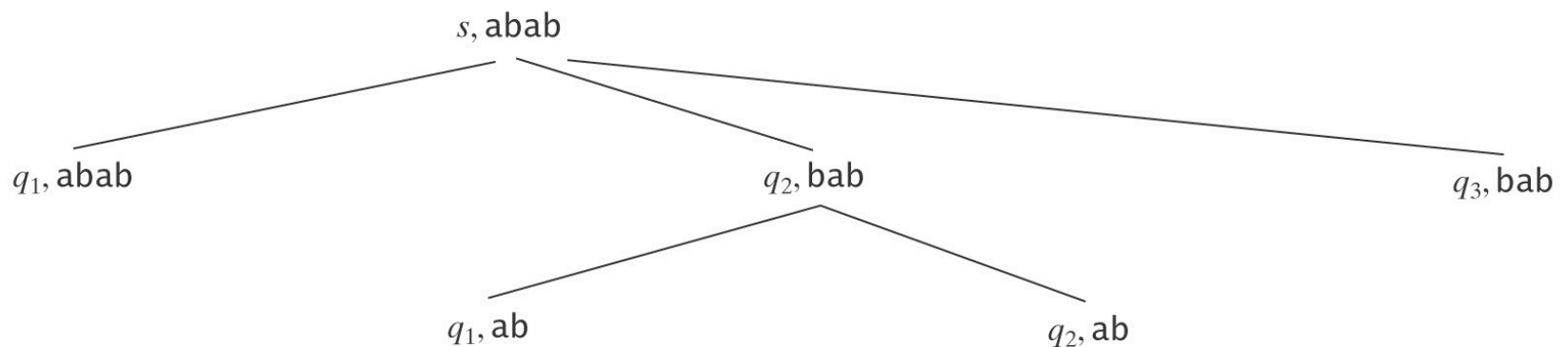
Sources of Nondeterminism



Analyzing Nondeterministic FSMs

Two approaches:

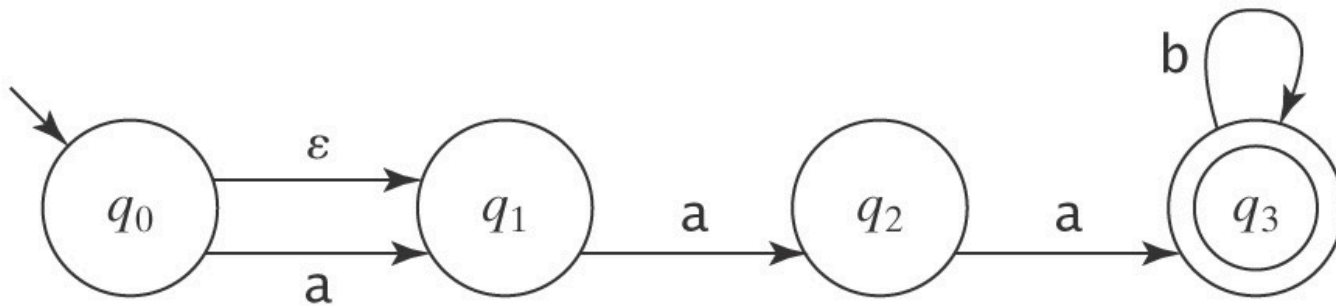
- Explore a search tree:



- Follow all paths in parallel

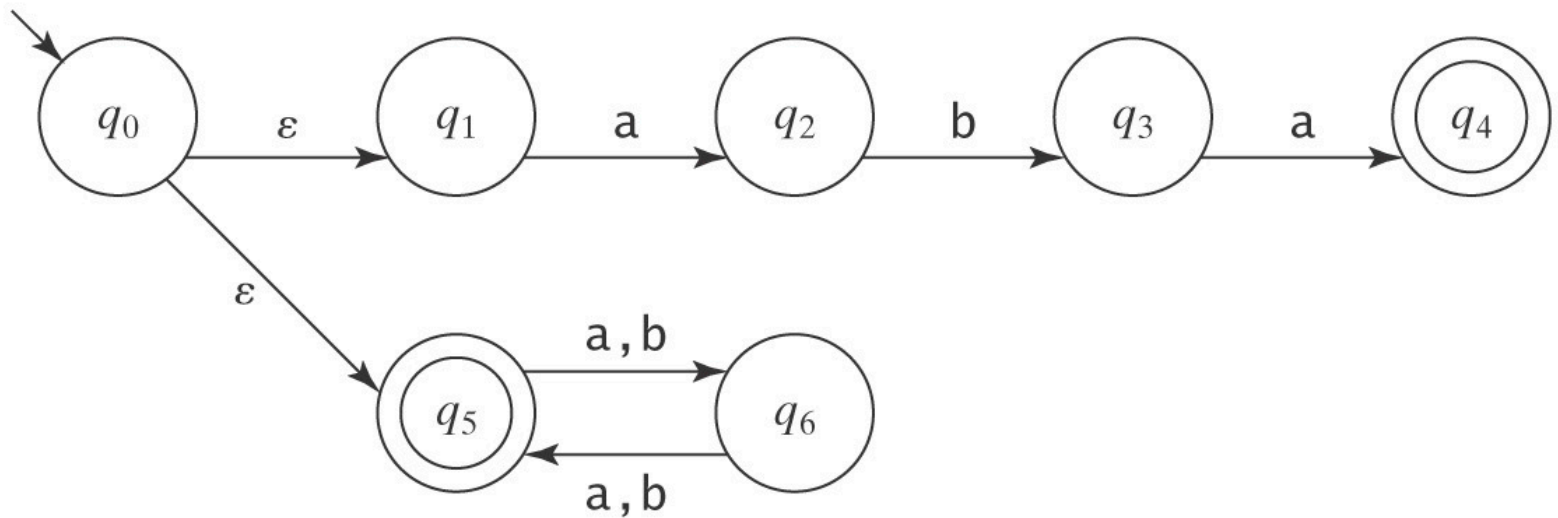
Optional Substrings

$L = \{w \in \{a, b\}^* : w \text{ is made up of an optional } a \text{ followed by } aa \text{ followed by zero or more } b\text{'s}\}.$



Multiple Sublanguages

$L = \{w \in \{a, b\}^* : w = aba \text{ or } |w| \text{ is even}\}.$

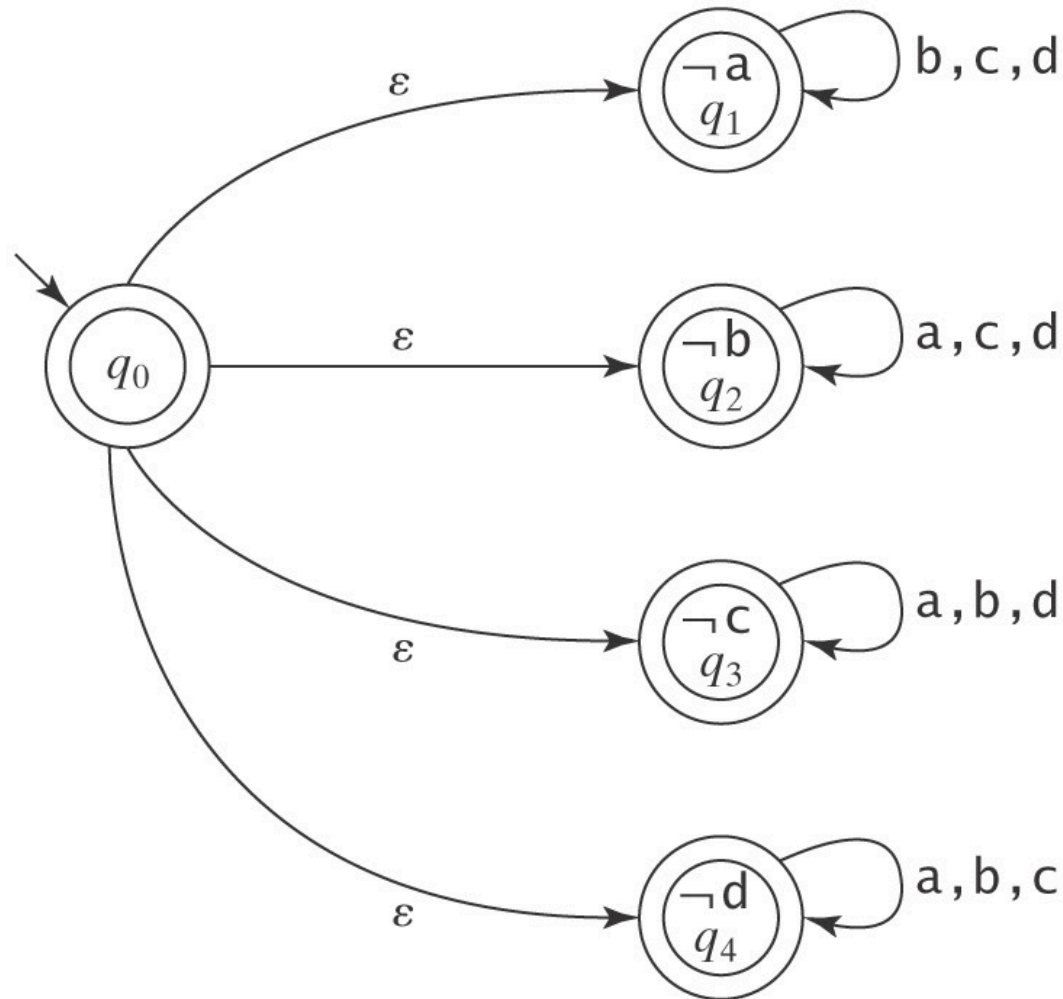




The Missing Letter Language

Let $\Sigma = \{a, b, c, d\}$. Let $L_{Missing} = \{w : \text{there is a symbol } a_i \in \Sigma \text{ not appearing in } w\}$

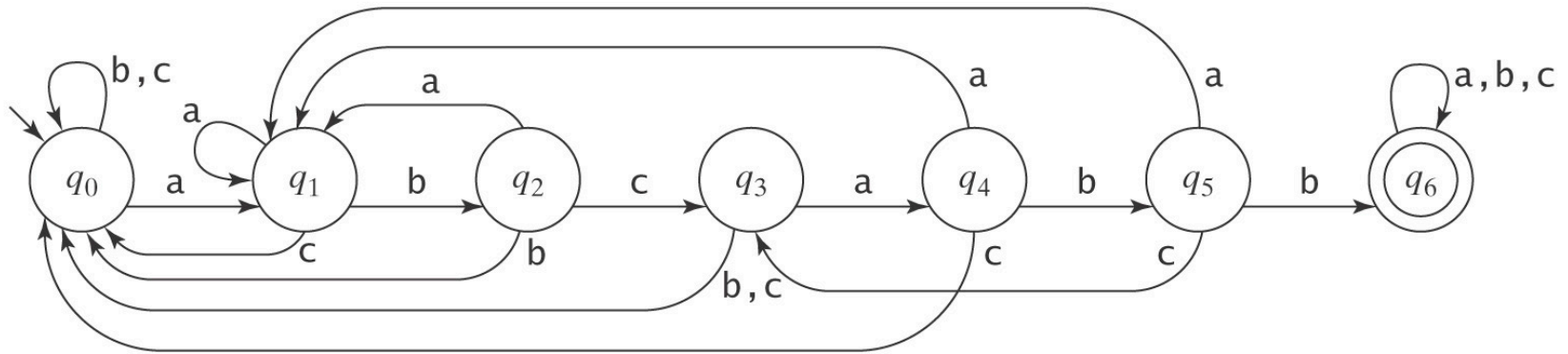
The Missing Letter Language



Pattern Matching

$$L = \{w \in \{a, b, c\}^* : \exists x, y \in \{a, b, c\}^* (w = x \text{ abcabb } y)\}.$$

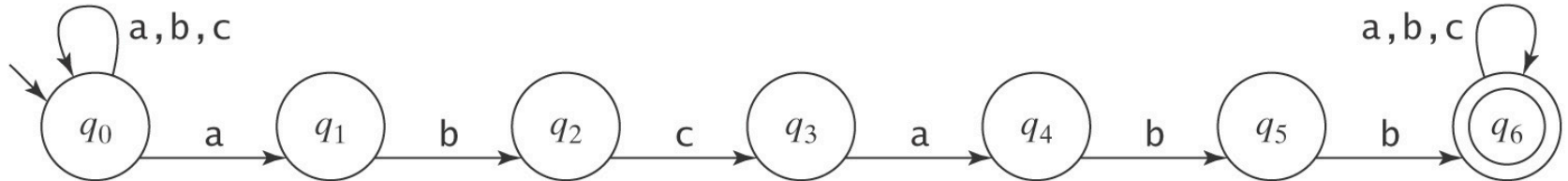
A DFSM:



Pattern Matching with NDFSMs

$$L = \{w \in \{a, b, c\}^* : \exists x, y \in \{a, b, c\}^* (w = x \text{ abcabb } y)\}.$$

An NDFSM:



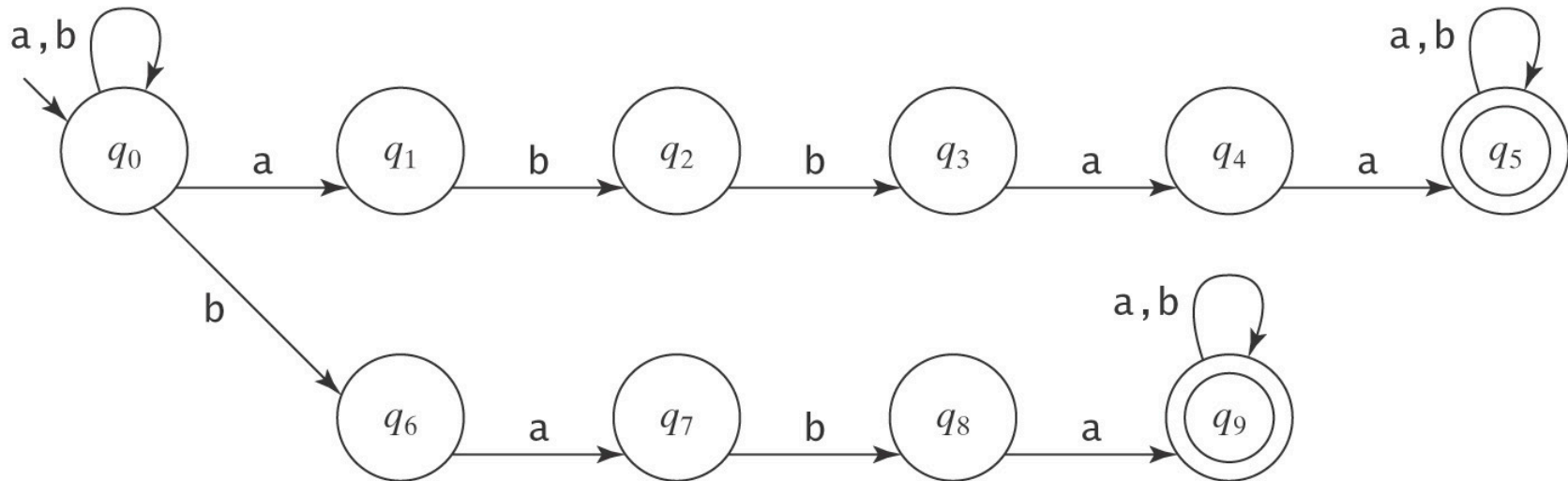


Multiple Keywords

$$L = \{w \in \{a, b\}^* : \\ \exists x, y \in \{a, b\}^* : w = x \text{ abbaa } y \text{ or } \\ w = x \text{ baba } y\}$$

Multiple Keywords

$$L = \{w \in \{a, b\}^* : \\ \exists x, y \in \{a, b\}^* : w = x \text{ abbaa } y \text{ or } \\ w = x \text{ baba } y\}$$



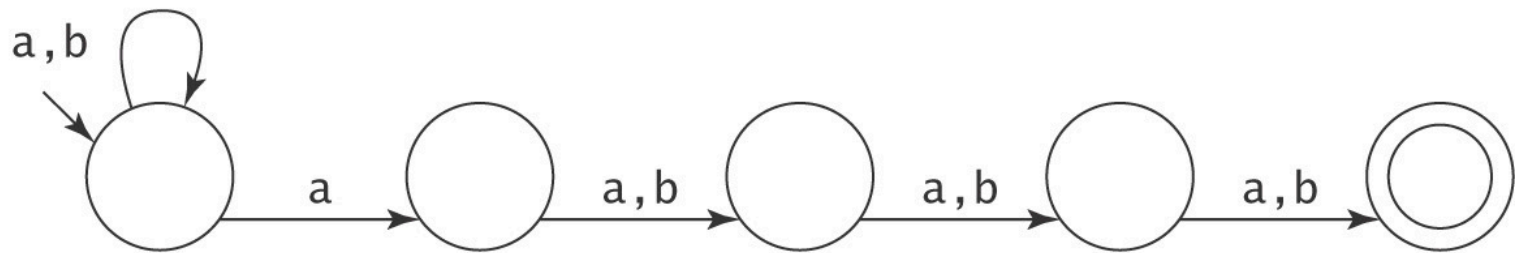


Checking from the End

$L = \{w \in \{a, b\}^* :$
the fourth to the last character is $a\}$

Checking from the End

$L = \{w \in \{a, b\}^* :$
the fourth to the last character is $a\}$





Another Pattern Matching Example

$L = \{w \in \{0, 1\}^* : w \text{ is the binary encoding of a positive integer that is divisible by 16 or is odd}\}$

Another NDFSM

$L_1 = \{w \in \{a, b\}^* : aa \text{ occurs in } w\}$

$L_2 = \{x \in \{a, b\}^* : bb \text{ occurs in } x\}$

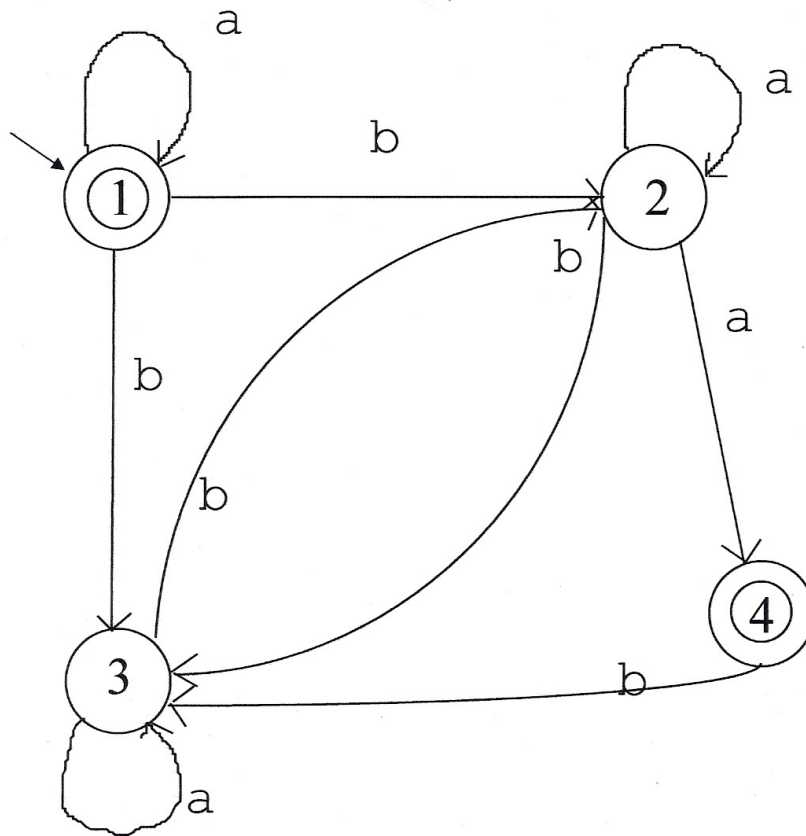
$L_3 = \{y \in \{a, b\}^* : y \in L_1 \text{ or } L_2\}$

$M_1 =$

$M_2 =$

$M_3 =$

Analyzing Nondeterministic FSMs



Does this FSM accept:

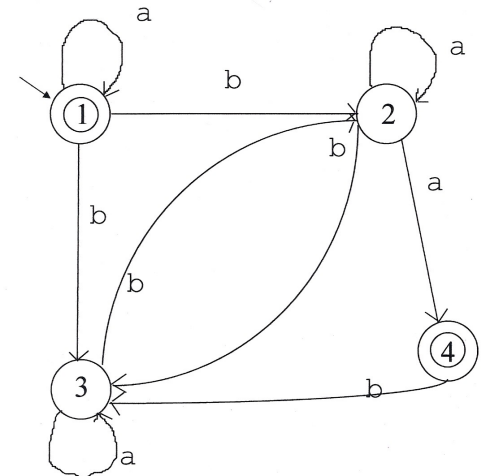
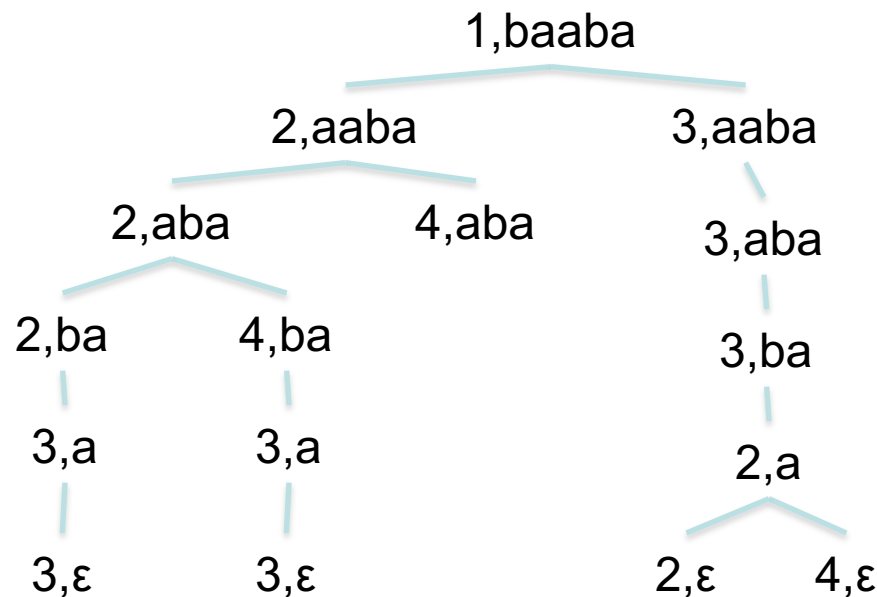
baaba

Remember: we just have to find one accepting path. 57

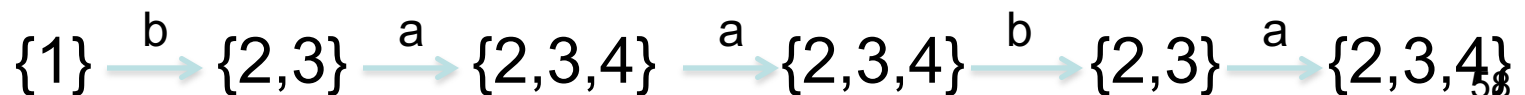
Analyzing Nondeterministic FSMs

Two approaches:

- Explore a search tree



- Follow all paths in parallel





Dealing with ε Transitions

ε -transitions change state without using input symbols

$$\textit{eps}(q) = \{p \in K : (q, w) \vdash_M^* (p, w)\}.$$

$\textit{eps}(q)$ is the closure of $\{q\}$ under the relation $\{(p, r) : \text{there is a transition } (p, \varepsilon, r) \in \Delta\}$.

How shall we compute $\textit{eps}(q)$?

An Algorithm to Compute $\text{eps}(q)$

Compute $\text{eps}(q: \text{state})$

$\text{result} = \{q\}$.

While there exists

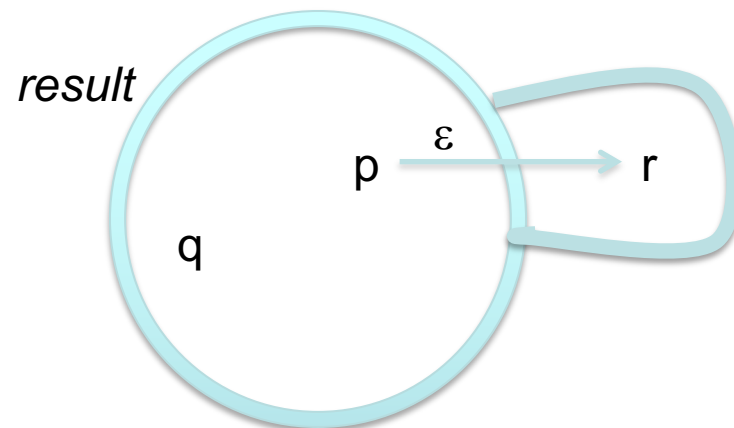
some $p \in \text{result}$ and

some $r \notin \text{result}$ and

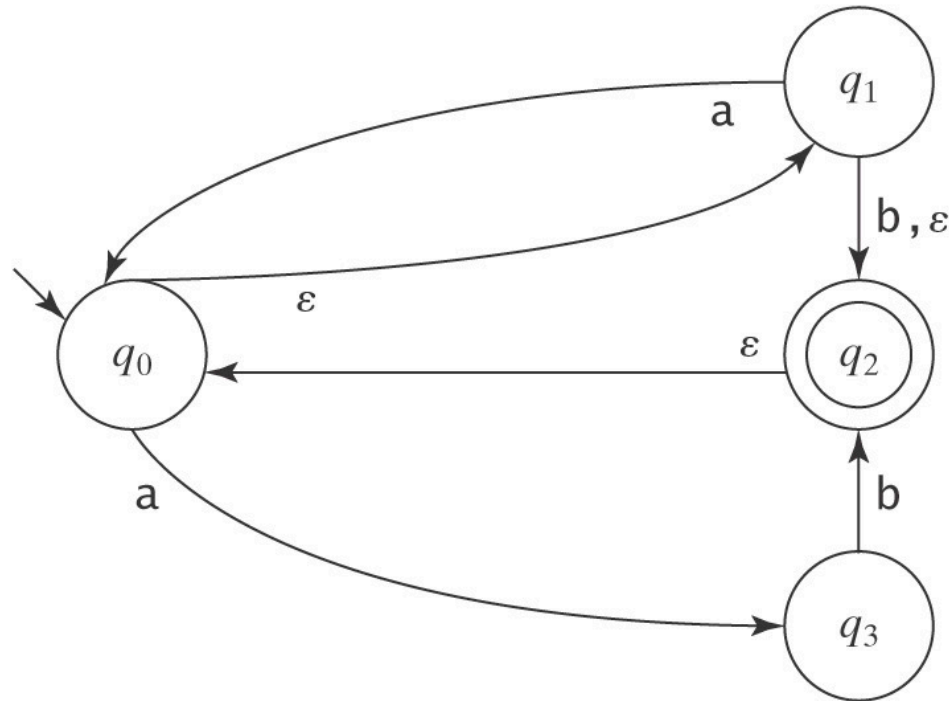
some transition $(p, \varepsilon, r) \in \Delta$ do:

Insert r into result .

Return result .



An Example of *eps*



$$eps(q_0) = \{q_0, q_1, q_2\}$$

$$eps(q_1) = \{q_0, q_1, q_2\}$$

$$eps(q_2) = \{q_0, q_1, q_2\}$$

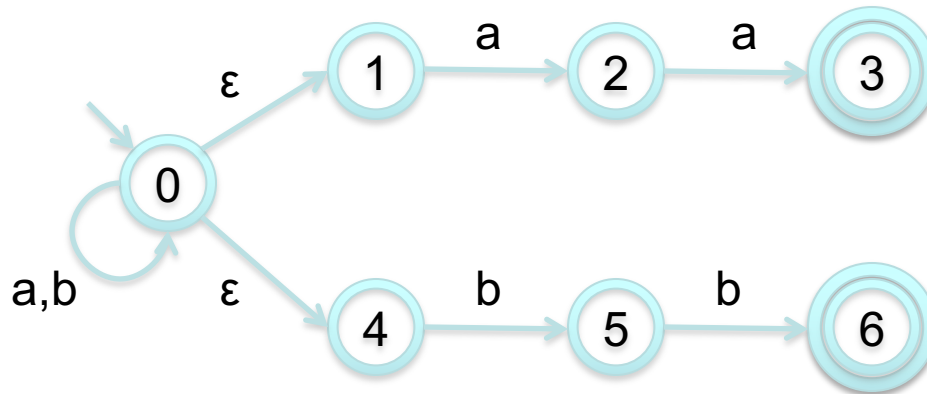
$$eps(q_3) = \{q_3\}$$

Simulating a NDFSM

ndfsmsimulate(*M*: NDFSM, *w*: string) =

1. *current-state* = *eps*(*s*).
2. For each *c* symbol of *w* do:
 1. *next-state* = $\emptyset$.
 2. For each state *q* in *current-state* do:
For each state *p* such that $(q, c, p) \in \Delta$ do:
next-state = *next-state* $\cup$ *eps*(*p*).
 3. *current-state* = *next-state*.
3. If *current-state* contains states in *A*, then accept.
else reject.

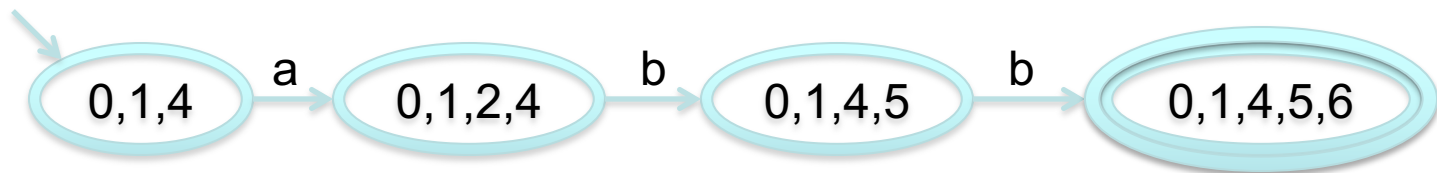
Example



$w = abb$

$\text{eps}(0) = \{0, 1, 4\}$

for $1 \leq i \leq 6$,
 $\text{eps}(i) = \{i\}$





Nondeterministic and Deterministic FSMs

Clearly: $\{\text{Languages accepted by a DFMS}\} \subseteq \{\text{Languages accepted by a NDFMS}\}$

More interestingly:

Theorem 5.3:

For each NDFMS, there is an equivalent DFMS.

Nondeterministic and Deterministic FSMs

Proof: By construction:

Given a NDFSM $M = (K, \Sigma, \Delta, s, A)$,
we construct $M' = (K', \Sigma, \delta', s', A')$, where

$$K' = \mathcal{P}(K)$$

$$s' = \text{eps}(s)$$

$$A' = \{Q \subseteq K : Q \cap A \neq \emptyset\}$$

$$\delta'(Q, a) = \bigcup \{ \text{eps}(p) : p \in K \text{ and } (q, a, p) \in \Delta \text{ for some } q \in Q \}$$



An Algorithm for Constructing the Deterministic FSM

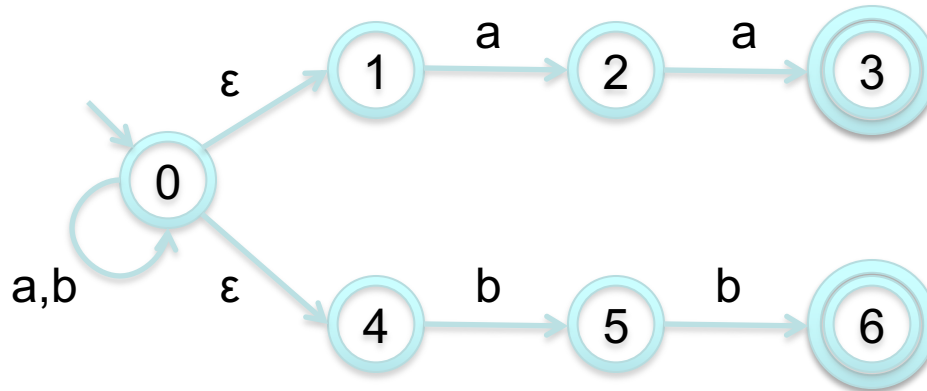
1. Compute the $eps(q)'$ s.
2. Compute $s' = eps(s)$.
3. Compute δ' .
4. Compute $K' =$ a subset of $\mathcal{P}(K)$.
5. Compute $A' = \{Q \in K' : Q \cap A \neq \emptyset\}$.

The Algorithm *ndfsmtodfsm*

ndfsmtodfsm(M : NDFSM) =

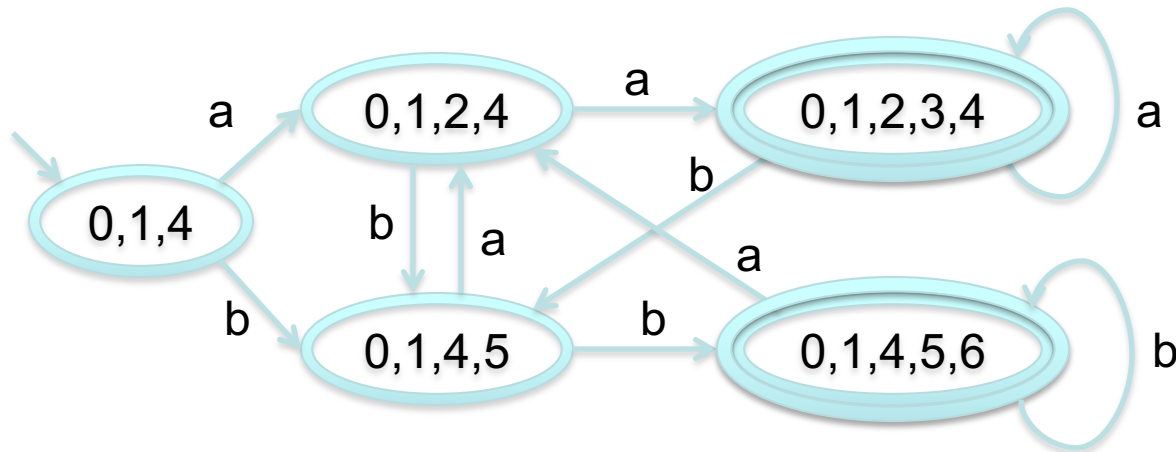
1. For each state q in K do:
 - 1.1 Compute $\text{eps}(q)$.
2. $s' = \text{eps}(s)$
3. Compute δ' :
 - 3.1 $\text{active-states} = \{s'\}$.
 - 3.2 $\delta' = \emptyset$.
 - 3.3 While $Q \in \text{active-states}$, $c \in \Sigma$ with $\delta'(Q, c)$ unknown do:
 $\text{new-state} = \emptyset$.
For each state q in Q do:
For each state p such that $(q, c, p) \in \Delta$ do:
 $\text{new-state} = \text{new-state} \cup \text{eps}(p)$.
 $\delta'(Q, c) = \text{new-state}$
 $\text{active-states} = \text{active-states} \cup \{ \text{new-state} \}$
4. $K' = \text{active-states}$.
5. $A' = \{Q \in K' : Q \cap A \neq \emptyset\}$.

Example



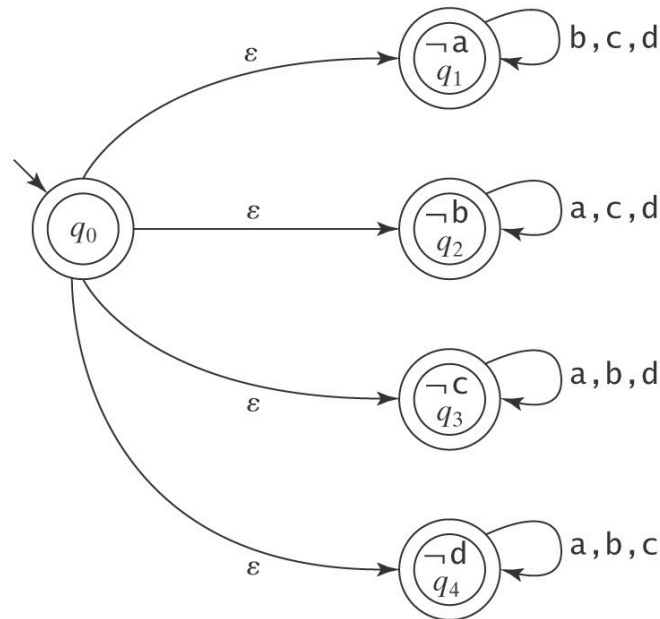
$\text{eps}(0) = \{0,1,4\}$

for $1 \leq i \leq 6$,
 $\text{eps}(i) = \{i\}$



The Number of States May Grow Exponentially

$$|\Sigma| = n$$



No. of states after 0 chars: $= 1$

No. of new states after 1 char: $\binom{n}{n-1} = n$

No. of new states after 2 chars: $\binom{n}{n-2} = n(n-1)/2$

No. of new states after 3 chars: $\binom{n}{n-3} = n(n-1)(n-2)/6$

Total number of states after $n-1$ chars: $2^n - 1$

Another Hard Example

$L = \{w \in \{a, b\}^* :$
the fourth to the last character is $a\}$

